Rapport TP1

ALAME FATIMA - SEMESTRE D'AUTOMNE 2026
Cours de mashups

Premièrement, je vous invite à plutôt regarder sur mon GitHub (https://github.com/fatimaalame/mashups_2026), où j'ai tout documenté et le readme explique mieux comment exécuter.

MON OBJECTIF:
L'objectif du TP1 est de développer un programme Python qui utilise au moins 3 fichiers de données distincts (dont au moins un JSON et un XML). J'ai pour ceci choisi d'aller chercher dans les bases de données OpenData des Transports publics genevois (TPG)[1]. Mon objectif est de pouvoir analyser les lignes du réseaux à partir des arrêts desservis, de la fréquentation des voyageurs et des km produits. Nous sommes aussi censé présenter 5 scénarios différents et une interface minimale qui s'affiche dans le terminal de l'IDE actuel.

SOURCES DE DONNÉES:
J'en ai 3, comme demandé dans les instructions. La première (frequentation.json) c'est les montées et descentes mensuelles par ligne et par arrêt[2]. La seconde (kilometres.json) c'est les km produits tous les jours par ligne.[3] Et pour la troisième, enfin un xml (arrets.xml), qui représente les arrêts, les quais et lignes desservies[4]. Pour faciliter et limiter les données, j'ai décidé que les calculs de fréquentation et de kilomètres sont effectués sur les données uniquement sur l'année 2025.

COMMENT EXECUTER ?
Pour lancer mon programme, il faut juste avoir Python 3 d'installé et j'ai utilisé uniquement des bibliothèques standard (json, pathlib, xml.etree.ElementTree) que vous devriez déjà avoir. Et donc une fois mon dossier téléchargé et décompressé, il suffit d'ouvrir un terminal, d'aller dans le dossier TP1 (là où se trouve mon fichier main.py) et de taper "python3 main.py". Là, le menu apparaîtra et il suffira de choisir entre le chiffre 1 et 5 pour tester les différents scénarios.
(P.s: Les trois fichiers de données sont déjà dans le sous-dossier data, donc pas besoin de télécharger quoi que ce soit d'autre.)

SCÉNARIO 1: FICHE D'UNE LIGNE TPG

```
    Recherche d'une ligne TPG

Entrez un numéro de ligne: 5
Nombre d'enregistrements trouvés: 789
Total des montées: 6375036.69
Total des descentes: 6375369.14
Nombre d'entrées d'arrêts : 66

Nombre d'arrêts distincts : 39

Liste des arrêts distincts :
- Chêne-Bougeries, Malagnou
- Chêne-Bougeries, Vallon
- Chêne-Bougeries,Coll.Claparède
- Genève, Appia
- Genève, Athénée
- Genève, Baulacre
- Genève, Bel-Air
- Genève, Bovy-Lysberg
- Genève, Contamines
- Genève, Coutance
- Genève, Florence
- Genève, Florissant
- Genève, Hôpital
- Genève, Intercontinental
- Genève, Motta
- Genève, Muséum
- Genève, Nations
- Genève, OMS-BIT
- Genève, Palais Eynard
- Genève, Philosophes
- Genève, Place Claparède
- Genève, Place de Neuve
- Genève, Rieu
- Genève, Varembé
- Genève, Vermont
- Genève, Vidollet
- Genève, Weber
- Genève, gare Cornavin
- Genève, poste
- Genève-Aéroport, Terminal
- Genève-Aéroport, gare-Arena
- Genève-Champel, gare/Peschier
- Grand-Saconnex, Crêts-Morillon
- Grand-Saconnex, Le Pommier
- Grand-Saconnex, Morillons
- Grand-Saconnex, Palexpo
- Grand-Saconnex, place
- Thônex, Sous-Moulin
- Thônex, Vallard


Nombre d'enregistrements kilométriques trouvés : 365
Total des kilomètres produits en 2025 : 845670.8
```

Capture d'écran de mon terminal pour le scénario 1, pour la ligne 5

Pour commencer, j'ai voulu faire quelque chose de simple. Je permet à l'utilisateur de choisir un ligne TPG et d'obtenir quelques informations dessus. Combien de personnes y sont montées ou descendues en 2025? Quels arrêts dessert-elles? Et combien de km a-t-elle parcouru? Et pour ça, j'ai combiné les 3 fichiers pour: Calculer le total des montées et descentes en 2025, Identifier les arrêts desservis par la ligne, Supprimer les noms d'arrêts en double, et enfin, calculer le total des kilomètres produits en 2025. C'est le seul scénario où j'utilise mes trois fichiers en même temps

Exemple avec la ligne 5 :

Le programme affiche également les noms des arrêts distincts par ordre alphabétique :)

Mon premier problème était que les numéros de ligne ne sont pas toujours écrits de la même manière dans les fichiers ("5" en JSON et "05" en XML). Donc j'ai utilisé "zfill(2)" pour uniformiser les numéros et pour pouvoir les comparer. Et aussi, le nombre de noms distincts ne correspond pas nécessairement au nombre exact d'arrêts physiques, donc je les ai aussi enlevés (les doublons). J'ai fais ça en utilisant set( ).

SCÉNARIO 2: TOP 5 DES ARRÊTS LES PLUS FRÉQUENTÉS D'UNE LIGNE TPG

Pour ce deuxième scénario, je voulais savoir quels étaient les arrêts les plus fréquentés d'une ligne. L'utilisateur peut donc choisir une ligne et mon programme va additionner les montées de 2025 pour afficher les 5 premiers. Je suis allée un peu plus loin en affichant aussi les codes des quais et leur coordonnées GPS (vu que certains arrêts ont 2 quais allant dans 2 directions différentes) J'ai du coup combiné 2 fichiers (frequentation.json et arrets.xml) pour : Récupérer tous les arrêts de la ligne choisie et additionner leurs montées sur l'année 2025. Classer les arrêts du plus fréquenté au moins fréquenté. Afficher uniquement les 5 premiers. Récupérer les codes des quais de ces arrêts dans le JSON. Chercher ces mêmes codes dans le XML pour retrouver leurs coordonnées GPS. Et enfin, signaler les codes qui n'ont pas été retrouvés dans le XML

Toujours un exemple avec la ligne 5 :

```
Votre choix : 2

   Top 5 des arrêts les plus fréquentés

Entrez un numéro de ligne: 5
Nombre d'arrêts trouvés : 54

Top 5 des arrêts de la ligne 05
- Gare Cornavin : 1136811.64 montées
- Genève-Aéroport-Terminal : 563675.37 montées
- Hôpital : 461987.32 montées
- Bel-Air : 456896.02 montées
- Coutance : 339810.3 montées

Codes des arrêts du top 5 :
Gare Cornavin : ['CVIN00', 'CVIN01', 'CVIN03']
  Quai CVIN01 : 46.2100977091878 6.14283748872439
  Quai CVIN03 : 46.2105159853196 6.14154115529418
  Codes absents du XML : ['CVIN00']
Genève-Aéroport-Terminal : ['AERO01']
  Quai AERO01 : 46.230964986137 6.11015922923408
Hôpital : ['HOPI01', 'HOPI03']
  Quai HOPI01 : 46.1946570704398 6.14893493682517
  Quai HOPI03 : 46.1947367634223 6.14945511149338
Bel-Air : ['BAIR00', 'BAIR04']
  Quai BAIR04 : 46.2042364950228 6.14205727348778
  Quai BAIR00 : 46.2050658366664 6.14297945041307
Coutance : ['COUT00', 'COUT01']
  Quai COUT01 : 46.207598749947 6.14236528922311
  Quai COUT00 : 46.2072529688423 6.14192240963702
```

Capture d'écran de mon terminal pour le scénario 2, pour la ligne 5

Le programme affiche aussi les codes des quais et leurs coordonnées GPS quand il arrive à les retrouver dans le XML :)
Enfait, les noms des arrêts ne sont pas toujours écrits de la même manière dans le JSON et le XML. Donc au lieu de les comparer directement par leur nom, j'ai utilisé les codes des quais pour faire la correspondance entre les deux fichiers. Par ex, pour Gare Cornavin, le JSON contient 3 codes : CVIN00, CVIN01 et CVIN03. Mais le programme ne retrouve que CVIN01 et CVIN03 dans le XML. J'ai donc ajouté une vérification pour afficher les codes absents. Et aussi, comme certains arrêts ont plusieurs quais, ils peuvent avoir plusieurs coordonnées GPS. Enfin, j'ai testé avec la ligne 9999, qui ne possède aucune donnée de fréquentation en 2025 dans mon fichier. Dans ce cas, le programme affiche simplement un message pour prévenir l'utilisateur au lieu de continuer avec un classement vide.

SCÉNARIO 3: TOP 10 DES LIGNES TPG AVEC LE PLUS D'ARRÊTS

Pour ce troisième scénario, j'ai voulu savoir quelles sont les lignes TPG qui desservent le plus d'arrêts. Cette fois, on ne part plus d'une ligne choisie par l'utilisateur mais on regarde direct l'ensemble du réseau. Mon programme va aller chercher dans les lignes du xml, compte leur noms d'arrêts et affiche les dix premiers. J'ai aussi rajouté le nombre de montées en 2025 pour avoir une idée de leur fréquentation et enrichir la recherche. Pour ce scénario, j'ai combiné 2 fichiers (arrets.xml et frequentation.json) pour récupérer toutes les lignes présentes dans le XML et les arrêts qu'elles desservent, enlever les noms d'arrêts en double avec set(), car un même arrêt peut avoir plusieurs quais, compter le nombre de noms d'arrêts distincts pour chaque ligne, classer les lignes de celle qui a le plus d'arrêts à celle qui en a le moins, afficher uniquement les 10 premières, et calculer le total des montées en 2025 pour chaque ligne à partir du JSON

Résultat trouvé:

```
Votre choix : 3

   Classement des lignes TPG

Nombre de lignes trouvées : 91

Top 10 des lignes avec le plus d'arrêts :
Ligne 01 : 44 arrêts distincts – 6892915.82 montées en 2025
Ligne 20 : 43 arrêts distincts – 3559197.45 montées en 2025
Ligne 54 : 43 arrêts distincts – 252972.95 montées en 2025
Ligne 38 : 43 arrêts distincts – 720297.71 montées en 2025
Ligne 05 : 39 arrêts distincts – 6375036.69 montées en 2025
Ligne A : 37 arrêts distincts – 1116000.14 montées en 2025
Ligne 83 : 35 arrêts distincts – 242246.12 montées en 2025
Ligne 60 : 35 arrêts distincts – 3180404.82 montées en 2025
Ligne 07 : 33 arrêts distincts – 5296553.26 montées en 2025
Ligne 09 : 33 arrêts distincts – 7162498.39 montées en 2025
```

Capture d'écran de mon terminal pour le scénario 3

Le programme a trouvé 91 lignes différentes dans le XML, mais j'ai décidé d'afficher uniquement les 10 premières pour que le résultat soit plus lisible dans le terminal.

Ce scénario m'a aussi permis de repérer un problème avec les lignes qui contiennent des lettres, comme la ligne A. Au début, je n'arrivais pas à retrouver leur fréquentation parce que j'avais prévu mon code surtout pour les numéros. Après avoir corrigé ma fonction de normalisation, j'ai finalement retrouvé les 1 116 000,14 montées de la ligne A en 2025.

SCÉNARIO 4: COMPARAISON DES HORAIRES VACANCES ET NORMAL

Pour ce 4eme scénario, je voulais voir s'il y avait une vraie différence entre les kilomètres produits pendant les jours avec un horaire normal et ceux avec un horaire vacances. Parce qu'en soit, on pourrait imaginer que les TPG font moins de kilomètres pendant les vacances, mais est-ce que c'est vraiment le cas ? Et surtout, de combien ? J'ai donc utilisé 2 fichiers (kilometres.json et frequentation.json) pour pouvoir comparer tout ça. En gros, le programme permet à l'utilisateur de choisir une ligne, puis il va chercher les kilomètres produits en 2025. Ensuite, j'ai séparé les données en deux catégories, VACANCES et NORMAL, pour calculer la moyenne des kilomètres produits pour chacune. Au début, je pensais simplement comparer le total des kilomètres, mais enfait ça n'aurait pas été très représentatif, parce qu'il n'y a pas le même nombre de jours dans les deux catégories. Donc j'ai plutôt décidé de calculer une moyenne par jour et ensuite de regarder la différence en pourcentage.

Par example avec la ligne 12:

```
Votre choix : 4

  Comparaison des horaires TPG

Entrez un numéro de ligne: 12
Jours distincts VACANCES : 63
Jours distincts NORMAL : 190
Enregistrements VACANCES : 63
Enregistrements NORMAL : 190

Moyenne VACANCES : 3717.38 km
Moyenne NORMAL : 3783.96 km
Différence : 66.58 km
Diminution par rapport à NORMAL : 1.76 %
Total des montées en 2025 : 21018435.15
```

Capture d'écran de mon terminal pour le scénario 4, avec la ligne 12

Pour la ligne 12, j'ai obtenu environ 3 717 km par jour en horaire VACANCES, contre 3 784 km en horaire NORMAL. Ça représente une diminution de seulement 1,76 %, ce qui est finalement assez faible. Et puisque la ligne 12 passe par Battelle, où se trouve notre cher CUI, je me suis dit que cette petite diminution était peut-être due aux étudiants qui ne viennent plus en cours pendant les vacances. Comme quoi, nos vacances font peut-être aussi du bien aux TPG, mais ce n'est qu'une hypothèse:)

SCÉNARIO 5: RECHERCHER UN ARRÊT TPG

Pour terminer, j'ai voulu faire un peu l'inverse du premier scénario. Cette fois, au lieu de chercher une ligne, l'utilisateur peut directement écrire le nom d'un arrêt. Le programme retrouve les quais correspondants, leurs coordonnées GPS, les lignes qui les desservent et le total des montées en 2025.

```
Votre choix : 5
    Recherche d'un arrêt TPG
Entrez le nom d'un arrêt : Battelle
Nombre d'entrées trouvées : 1

Arrêt : Carouge GE, Battelle
Quai : BALE00
Coordonnées : 46.178846923896 6.13946783276305

Lignes desservant cet arrêt : ['41']
Total des montées en 2025 : 1184.97
```

Capture d'écran de mon terminal pour le scénario 5, avec l'arrêt "Battelle"

Pour cet exemple, j'ai choisi l'arrêt Battelle et j'ai retrouvé un quai (BALE00), desservi par la ligne 41, avec un total de 1 184,97 montées en 2025. J'ai aussi fait en sorte qu'on puisse écrire seulement une partie du nom, sans forcément connaître son nom complet. Par exemple, « Battelle » suffit pour retrouver « Carouge GE, Battelle ».

CONCLUSION

Ce TP m'a permis de comprendre que la difficulté ne réside pas uniquement dans la programmation, mais aussi dans la manière dont on exploite et interprète les données. En travaillant avec trois sources différentes, j'ai été confrontée à plusieurs problèmes de correspondance, de cohérence et de traitement des données. J'ai notamment appris qu'obtenir un résultat ne suffit pas. Encore faut-il comprendre ce qu'il représente et s'assurer qu'il est pertinent. Au-delà des cinq scénarios réalisés, ce projet m'a surtout appris à questionner mes résultats, à identifier mes erreurs et à améliorer progressivement mon programme. Finalement, ce que je retiens de ce TP, c'est qu'un programme qui fonctionne n'est pas nécessairement un programme qui donne du sens aux données. Et c'est justement cette distinction qui m'a le plus appris.

UTILISATION DE L'INTELLIGENCE ARTIFICIELLE

J'ai utilisé l'IA pour m'aider à comprendre et combiner mes fichiers JSON et XML, corriger mes erreurs Python (et il y en a eu pas mal !), vérifier mes résultats et m'assurer que mes solutions correspondaient au niveau attendu pour ce TP. Elle m'a aussi beaucoup aidée à structurer et reformuler ma documentation, notamment le README et certaines parties de ce rapport, tout en gardant mes propres explications et observations. J'ai tenu à effectuer les corrections moi-même, étape par étape pour comprendre ce que je faisais plutôt que de simplement reprendre du code généré. Les outils que j'ai utilisé sont GPT-5.5 Thinking (OpenAI) et Claude Sonnet 5.5 (Anthropic).

SOURCES UTILISÉES POUR MON CODE

- Documentation du module JSON: https://docs.python.org/fr/3/library/json.html [consulté le 24.10.26]
- Documentation du module ElementTree: https://docs.python.org/fr/3/library/xml.etree.elementtree.html [consulté le 24.10.26]
- Pour la structure de données: https://docs.python.org/fr/3/tutorial/datastructures.html [consulté [24.10.26]
- Pour le path: https://docs.python.org/fr/3/library/pathlib.html [consulté le 24.10.26]
- Pour les fonctions pour mes chaines de caractères: https://docs.python.org/fr/3/library/stdtypes.html#text-sequence-type-str [consulté le 24.10.26]
- Pour les fichiers XML: https://docs.python.org/fr/3/library/xml.etree.elementtree.html [consulté le 25.10.26]
- Pour la normalisation: https://www.geeksforgeeks.org/python/python-string-zfill/ [consulté le 27.10.26]
- Pour le set ( ) : https://www.geeksforgeeks.org/python/python-set-function/ [consulté le 27.10.26]

SOURCES POUR MES DONNÉES

[1] https://www.tpg.ch/en/open-data

[2] https://opendata.tpg.ch/explore/dataset/montees-mensuelles-par-arret-par-ligne/table/?disjunctive.ligne&disjunctive.ligne_type_act&disjunctive.arret&disjunctive.arret_code_long

[3] https://opendata.tpg.ch/explore/dataset/kilometres-produits-journaliers-par-ligne/export/?disjunctive.ligne&disjunctive.ligne_type_act&disjunctive.horaire_type&disjunctive.jour_semaine&sort=date&refine.date=2025

[4] https://opendata.swiss/fr/dataset/reseau-tpg-arrets1?sort=title-asc